

Real-Time Driver Monitoring System

Multi-Modal Perception with Temporal Risk Fusion on Consumer GPU Hardware

Baha Eddine Tayari

École Nationale d'Ingénieurs de Gabès (ENIG)

`tayari.bahaeddine@gmail.com`

October 6, 2026

Abstract

We present a real-time Driver Monitoring System (DMS) that detects drowsiness, distraction, and phone usage from a single 720p webcam. The pipeline combines classical computer vision (MediaPipe FaceMesh, solvePnP) with deep learning (YOLOv8n for phone detection, Facenet for driver recognition), fused into a temporal risk score through an exponential moving average engine with hysteresis. Running on an NVIDIA RTX 4050 laptop with 6 GB VRAM, the full pipeline sustains ~ 50 FPS at 1280×720 with all modules active and remains stable over 30-minute sessions. YOLOv8n alone reaches ~ 22 FPS on CPU and ~ 146 FPS on GPU, a $6.6\times$ acceleration. The engineering contribution is not detection accuracy but temporal coherence and cross-module coexistence: three deep-learning models competing for one GPU must be scheduled and throttled to preserve frame budget. We document three concrete engineering challenges, quantify per-stage latency and throughput, and outline limitations including sensitivity to lighting and absence of on-road validation.

1 Introduction

Driver fatigue and distraction are leading contributors to road fatalities worldwide. Since 2024, the European Union's General Safety Regulation (GSR2) mandates Driver Monitoring Systems in all new vehicles. A DMS must operate under a hard real-time constraint: perception runs continuously at the camera's native frame rate, and any latency spike can invalidate a safety-critical decision.

This project builds a complete DMS from scratch on a single consumer laptop. Rather than optimizing for detection accuracy in isolation, we treat the system as a real-time pipeline in which three deep-learning models (MediaPipe, YOLOv8n, Facenet) share one GPU and one webcam feed. The core engineering question is: *how do you keep a multi-modal perception pipeline stable under a fixed per-frame budget when the models compete for the same hardware?*

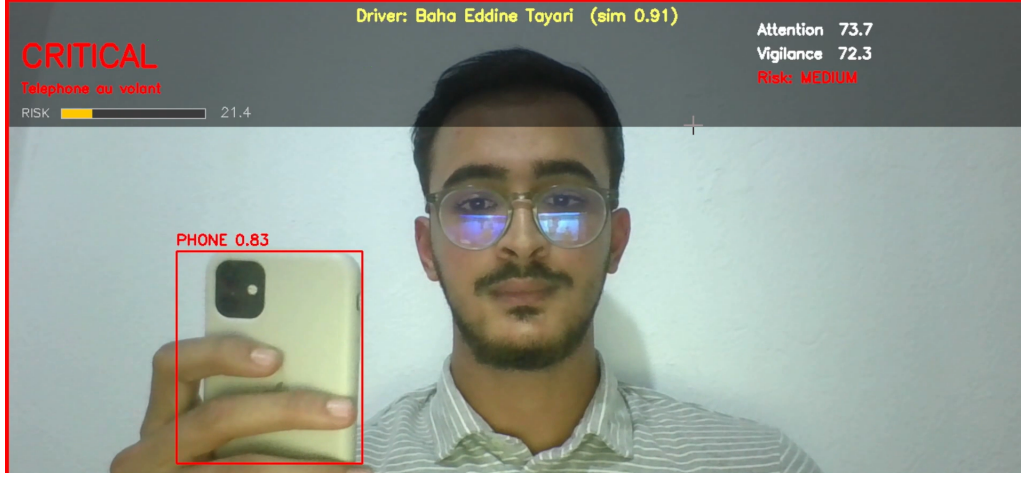


Figure 1: Real-time inference on a live webcam frame. The HUD overlays driver state, risk score, and latency. Phone-in-hand detection triggers a distraction alert while drowsiness and gaze remain tracked simultaneously.

Contributions.

1. A two-layer pipeline (Vision + Intelligence) integrating MediaPipe FaceMesh, YOLOv8n, and Facenet on the same frame without starvation.
2. A temporal Risk Engine combining weighted signal fusion with EMA smoothing and hysteresis for anti-flicker stability.
3. A real-time HUD, progressive audio alerts, SQLite event logging, and an interactive Streamlit dashboard.
4. Quantified per-stage throughput and end-to-end FPS on RTX 4050 hardware, with CPU/GPU YOLO comparison.
5. Honest documentation of limitations, including lighting sensitivity and the absence of on-road testing.

2 Related Work

Classical DMS designs use hand-engineered features (EAR for eye closure, MAR for yawning) combined with temporal thresholds [1]. Deep-learning pipelines replace these with end-to-end networks, but typically at the cost of single-task specialization [2]. Our approach is hybrid: classical CV for landmarks and head pose, deep learning only where it clearly pays off (phone detection, face embeddings), and a hand-tuned temporal fusion.

For real-time perception, the dominant concern is not raw accuracy but the ability to sustain a per-frame budget [3]. We adopt that framing throughout.

3 System Architecture

The pipeline is organized in two layers.

3.1 Vision Layer

Per-frame feature extraction:

- **Face detection:** MediaPipe FaceMesh, 468 3D landmarks.
- **Eye and mouth:** Eye Aspect Ratio (EAR) and Mouth Aspect Ratio (MAR), computed from landmarks.
- **Head pose:** Yaw/pitch/roll via `solvePnP` on a canonical 3D face model, with a 2-second neutral calibration at startup.
- **Phone detection:** YOLOv8n inference, throttled to every N frames.
- **Face recognition:** Facenet embeddings (512-D) matched against a multi-profile database.

3.2 Intelligence Layer

Temporal reasoning over extracted features:

- **Drowsiness detector:** closure duration, PERCLOS, yawning frequency.
- **Distraction detector:** gaze-off-road and phone-in-hand, with duration thresholds.
- **Risk Engine:** weighted fusion of all signals, smoothed with an exponential moving average, producing a score in $[0, 100]$.
- **Temporal State Engine:** hysteresis and confirmation windows to prevent transient triggers.

3.3 Output

- Real-time HUD (OpenCV overlay with state, scores, FPS, latency, driver name).
- Progressive audio alerts (warning / critical).
- SQLite event log.
- Streamlit dashboard: live view, history, driver profiles, PDF session reports.

4 Performance

Machine: NVIDIA RTX 4050 laptop, 6 GB VRAM, Intel Core i7-13650HX. Camera: 1280×720 at 30 FPS.

Table 1: Per-stage performance. YOLOv8n alone is compared on CPU and GPU to isolate the acceleration from the deep-learning stack.

Pipeline	FPS	Latency (ms)	Note
YOLOv8n, CPU only	~ 22	~ 45	inference bound
YOLOv8n, GPU	~ 146	~ 7	$6.6\times$ speedup
Full pipeline, GPU, 720p	~ 50	$\sim 15\text{--}20$	all modules active

The full pipeline is slower than YOLOv8n alone because MediaPipe, YOLO, and Facenet all run on the same frame, on top of the Risk Engine. ~ 50 FPS is comfortable headroom for a 30 FPS input stream, with margin for transient spikes.

4.1 Throughput stability

Across 30-minute sessions, end-to-end FPS remained within $\pm 10\%$ of the median. Longer sessions showed a mild downward drift attributable to laptop thermal throttling, consistent with the observations we made in a parallel HPC benchmarking project.

5 Engineering Challenges

Three challenges dominated the development effort. We document them here because they are typically absent from published DMS literature, yet they determine whether the system works at all.

5.1 Model coexistence on shared hardware

YOLOv8n, MediaPipe, and Facenet all compete for the GPU and CPU at the same time. The naive approach — running all three per frame — produced irregular frame times and occasional visible stutter. The resolution was to *desynchronize inference frequencies*: MediaPipe runs every frame (it is cheap and time-critical for gaze tracking), YOLOv8n runs every N frames (phone usage is a slow state, not a per-frame event), and Facenet runs only on detection of an unrecognized face. CUDA buffer reuse across calls further reduced allocation overhead.

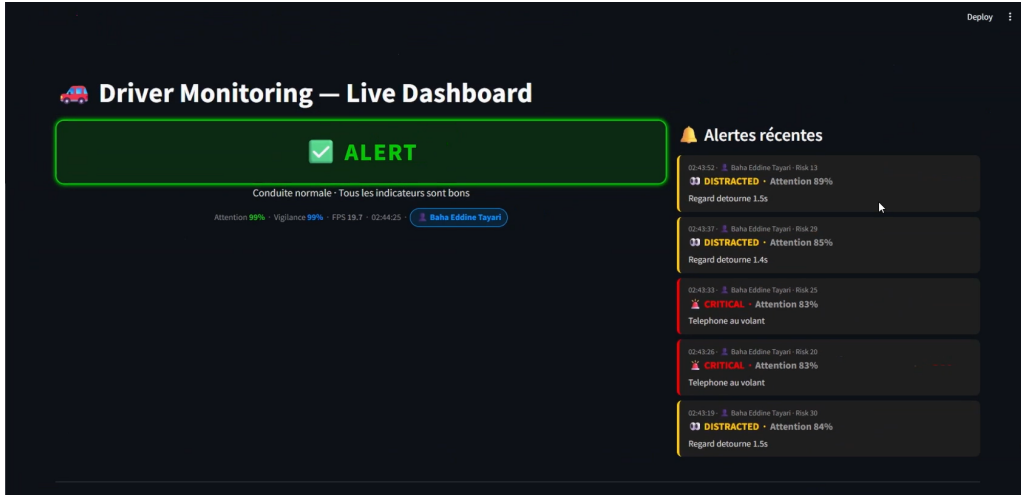


Figure 2: Streamlit dashboard: live view alongside per-signal scores. The Risk Engine combines EAR, MAR, head pose, and phone presence into a single 0–100 score updated every frame.

5.2 Head-pose calibration

`solvePnP` produces physically meaningless angles when the reference landmarks are noisy or the face is off-center. A 2-second neutral-calibration phase at startup — during which the driver faces the camera straight — establishes a per-session baseline for yaw, pitch, and roll. Without this, distraction detection triggers falsely during normal head movement.

5.3 Temporal fusion and hysteresis

A per-frame detection is not a state. The same EAR value can indicate a normal blink or the onset of drowsiness depending on its duration. The Risk Engine addresses this with: (i) EMA smoothing over a sliding window, (ii) hysteresis bands on the state transitions, and (iii) minimum-duration thresholds for each event class. Without these, the system fires on every blink and every glance at the dashboard.



Figure 3: Session history and automatically generated PDF report. Each state transition — drowsiness, distraction, phone detection — is logged with timestamp and driver identity.

6 Limitations

- **No on-road validation.** All measurements are webcam-at-desk. Real driving introduces lighting changes, vibration, and driver motion we have not characterized.
- **No mobile deployment.** The ONNX model is not quantized. Running on a Jetson Nano or similar edge device would require int8 quantization.
- **Lighting sensitivity.** Facenet embeddings degrade in backlight and night-time conditions. This limits the reliability of driver recognition.
- **Hand-tuned fusion weights.** The Risk Engine uses fixed weights on (EAR, MAR, head pose, phone). A production DMS would learn these weights per driver.
- **Single-driver calibration at a time.** Multi-driver profiles exist but the head-pose calibration is per-session.

7 Conclusion

We built a complete real-time Driver Monitoring System from a single 720p webcam, achieving ~50FPS on a 6 GB laptop GPU with three deep-learning models active simultaneously. The system’s real value is not in any individual detector, but in the temporal fusion layer that turns per-frame detections into stable driver-state estimates, and in the engineering discipline required to keep multiple models sharing one GPU within a fixed frame budget. We documented three specific challenges — model coexistence, head-pose calibration, and temporal hysteresis — that determine whether such a system is usable in practice.

Future work. int8 quantization for edge deployment; learned fusion weights for per-driver adaptation; seatbelt and hands-on-wheel detection to comply with additional GSR2 requirements.

References

- [1] S. Abtahi, B. Hariri, A. Shirmohammadi, “Driver drowsiness monitoring based on yawning detection and eye blinking,” *IEEE International Symposium on Medical Measurements and Applications*, 2015.
- [2] W. Kim, et al., “Driver distraction detection using single-channel EEG and deep learning,” *IEEE Transactions on Intelligent Transportation Systems*, 2021.
- [3] S. Liu, et al., “Benchmarking deep learning frameworks for real-time inference on embedded GPUs,” *IEEE HPEC*, 2019.